PANDUAN BUILD WORKSHOP

Bangun AI Project Manager Anda

Referensi bawa-pulang: konsep, tiap langkah kode, dan cara menjalankannya — CrewAI + Ollama lokal.

Ini pendamping sesi praktik dan referensi Anda setelahnya. Berkas yang bisa dijalankan (pm_crew.py, virtual_office.py) ada di bundel kode; panduan ini menjelaskan apa yang dilakukan tiap langkah dan mengapa.

Model mental

- **Agent —** sebuah LLM plus tools, memory, dan sebuah loop. Tanpa hype, hanya itu.
- **Loop ReAct —** tiap putaran agent berpikir (Thought), bertindak/Action (memanggil tool), mengamati/Observe hasilnya, dan mengulang hingga tujuan tercapai. CrewAI menjalankan loop ini untuk Anda.
- **Crew —** satu agent untuk tugas terfokus; satu crew spesialis untuk pekerjaan kompleks. Sequential = pipeline; hierarchical = manager mendelegasikan.
- **Stack lokal —** CrewAI mengorkestrasi, LiteLLM berbicara ke model apa pun, Ollama menjalankan open weights (Llama 3.1 / Qwen 2.5) di mesin Anda. Tanpa API key, tanpa data keluar dari laptop.

Setup

Instal model lokal sekali, buat environment Python bersih, lalu tambahkan framework-nya.

```
# 1. install Ollama (ollama.com), then pull open models
ollama pull llama3.1          # ~4.7 GB, runs fully local
ollama pull qwen2.5           # a strong alternative

# 2. a clean Python environment + the framework
python -m venv .venv && source .venv/bin/activate
pip install crewai crewai-tools pydantic

# 3. sanity check: does the local model answer?
ollama run llama3.1 "reply with one word: ready"
```

Membangun asisten PM, langkah demi langkah

1. Agent pertama Anda

Sebuah Agent, sebuah Task, dan sebuah Crew untuk menjalankannya — diarahkan ke model Ollama lokal.

```
from crewai import Agent, Task, Crew, Process, LLM


# point CrewAI at the LOCAL model served by Ollama
llm = LLM(model="ollama/llama3.1",
          base_url="http://localhost:11434")


planner = Agent(
    role="Project Planner",
    goal="Turn a project brief into a clear task list",
    backstory="A sharp PM who makes vague briefs actionable.",
    llm=llm, verbose=True,
)


task = Task(
    description="Break down this brief into tasks:\n{brief}",
    expected_output="A numbered task list with an owner each",
    agent=planner,
)


crew = Crew(agents=[planner], tasks=[task], process=Process.sequential)
print(crew.kickoff(inputs={"brief": open("brief.md").read()}))
```

Mengapa: Satu agent + satu task + satu crew adalah asisten berfungsi dalam ~20 baris.

2. Beri sebuah tool

Tool hanyalah fungsi Python. Jika model memutuskan membutuhkannya, CrewAI memanggilnya dan mengembalikan hasilnya.

```
from crewai.tools import tool
import json


@tool("save_tasks")
```

```
def save_tasks(tasks_json: str) -> str:
    "Save a JSON task list to project_state.json."
    data = json.loads(tasks_json)
    with open("project_state.json", "w") as f:
        json.dump(data, f, indent=2)
    return f"saved {len(data)} tasks"


# the agent may now decide to call this tool
planner.tools = [save_tasks]
```

Mengapa: Ekspos hanya tool yang benar-benar diperlukan agent (least privilege).

3. Output bertipe, bukan prosa

Buat planner mengembalikan objek Pydantic tervalidasi agar langkah berikutnya bisa memercayai datanya.

```
from pydantic import BaseModel
from typing import List


class TaskItem(BaseModel):
    title: str
    owner: str
    estimate_days: int


class Plan(BaseModel):
    tasks: List[TaskItem]


plan_task = Task(
    description="Break the brief into tasks:\n{brief}",
    expected_output="A Plan object",
    agent=planner,
    output_pydantic=Plan,     # validated, typed output
)
```

Mengapa: Output bertipe membuat agent bisa dirangkai — tanpa regex pada prosa.

4. Tambah dua spesialis

Seorang risk analyst dan seorang status writer, masing-masing satu tanggung jawab sempit.

```
risk = Agent(
    role="Risk Analyst",
    goal="Flag the top risks, with one mitigation each",
    backstory="A cautious PM who has watched projects slip.",
    llm=llm)

writer = Agent(
    role="Status Writer",
    goal="Draft a short status update for stakeholders",
    backstory="Writes clear, calm updates.",
    llm=llm)

risk_task = Task(description="Review the plan. List the top risks.",
                 expected_output="A risk register", agent=risk)

status_task = Task(description="Write a status update from the plan.",
                   expected_output="A 5-line update", agent=writer)
```

Mengapa: Operkan konteks antar-task dengan context=[...] agar tiap agent melihat output sebelumnya.

5. Jalankan PM crew

Rangkai tiga agent menjadi satu sequential crew lalu jalankan pada brief nyata.

```
crew = Crew(
    agents=[planner, risk, writer],
    tasks=[plan_task, risk_task, status_task],
    process=Process.sequential,     # planner -> risk -> writer
    verbose=True,
)
```

```
result = crew.kickoff(inputs={"brief": open("brief.md").read()})

print(result)      # tasks + risk register + a status update
```

Mengapa: Itulah AI project manager Anda — sekitar 40 baris.

Intermezzo: kantor virtual (NexaPaw)

Beralih dari pipeline tetap ke seorang Lead bernama yang mendelegasikan ke tim spesialis — pola di balik kantor NexaPaw milik Felix, di mana tiap Claude Code role adalah agent (CEO dan CTO di atas para project lead di atas para engineer spesialis), divisualisasikan live di HQ 3D. Anda definisikan staf; Lead menentukan siapa mengerjakan apa, dan kapan.

```
# a NexaPaw-style project team: a Lead delegates to specialists

lead = Agent(role="Project Lead",

             goal="Deliver the feature; assign work to the right role",

             backstory="Runs a NexaPaw project team.", llm=llm)


cv  = Agent(role="CV Engineer", goal="Build + evaluate the model", llm=llm)

be  = Agent(role="Backend Engineer", goal="APIs and data", llm=llm)

fe  = Agent(role="Frontend Engineer", goal="The user interface", llm=llm)

qa  = Agent(role="QA Engineer", goal="Test against the spec", llm=llm)


office = Crew(

    agents=[cv, be, fe, qa],

    tasks=[deliver_feature],        # one high-level goal

    process=Process.hierarchical,   # the Lead delegates

    manager_agent=lead,             # a named manager, not just an LLM

    verbose=True,

)

office.kickoff(inputs={"brief": "Ship cross-camera re-ID v2"})
```

TRADE-OFF JUJUR

Kantor virtual itu kuat dan lebih sulit dikendalikan. Lebih banyak agent = lebih banyak panggilan model = lebih lama dan nondeterministik. Pakai hanya saat satu agent mentok.

Membuatnya nyata

Validasi output, batasi iterasi, dan tahan tindakan berdampak di belakang manusia.

```
from pathlib import Path


# gate risky actions behind a human — draft, never send
@tool("send_email")
def send_email(to: str, body: str) -> str:
    "Save a draft for human approval; does NOT send."
    Path("outbox.md").write_text(body)
    return "DRAFT saved to outbox.md — needs approval"


# stop a stuck agent from looping forever
planner = Agent(role="Project Planner", goal="...",
                llm=llm, max_iter=8, max_retry_limit=2)
```

- Aktifkan verbose / logging untuk melihat tiap thought, panggilan tool, dan hasil.
- Lacak token dan latensi — model lokal gratis tetapi lebih lambat; cache panggilan berulang.
- Evaluasi pada set brief tetap dengan output yang diketahui benar sebelum dipercaya.
- Local-first menjaga data pelanggan dari server pihak ketiga (membantu UU PDP).
- Waspadai prompt injection saat agent membaca teks eksternal, email, atau halaman web.

ATURAN SEDERHANA

Agent sebaiknya mengusulkan; manusia yang memutuskan — untuk hal apa pun yang penting (kirim, bayar, hapus, atau keputusan tentang orang).

Jalankan kode jadinya

- ollama pull llama3.1 (sekali)
- pip install -r requirements.txt
- python pm_crew.py --brief brief.md — merencanakan tugas, menandai risiko, menyusun status update
- python virtual_office.py — kantor virtual hierarchical
- Jika import atau argumen gagal, cek versi CrewAI terpasang terhadap dokumentasinya saat ini.

Disusun oleh Felix Yustian Setiono untuk MurzEdu Program (ecommurz). Kode adalah titik awal — sesuaikan dengan stack Anda.